Contents

Union-Find (Disjoint Set Union) — Complete Interview Handbook

Pattern #19 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Graph section

---

Table of Contents

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

Union-Find (Disjoint Set Union, DSU) maintains a collection of disjoint (non-overlapping) sets, supporting two operations extremely efficiently: `find(x)` — determine which set element `x` belongs to (via a representative/root), and `union(x, y)` — merge the sets containing `x` and `y`. With two optimizations (union by rank/size and path compression), both operations run in **amortized nearly-O(1)** time — technically O(α(n)), where α is the inverse Ackermann function, which grows so slowly it's effectively constant for any realistic input size.

1.2 Why Was This Pattern Invented?

Repeatedly answering "are these two elements connected?" via BFS/DFS from scratch for every query is wasteful when edges are added incrementally over time (dynamic connectivity) — each query would cost O(V+E). Union-Find was invented to **maintain connectivity incrementally**: as edges are added one at a time, each "are they connected" or "connect them" operation costs nearly O(1) amortized, rather than re-running a full traversal.

1.3 Real Intuition Behind The Pattern

Imagine **merging companies through acquisitions** — each company starts as its own "team," and whenever company A acquires company B, everyone in B now reports up to A's ultimate parent company. To check "do these two employees work for the same ultimate parent company," you just trace each one up their reporting chain to the top and compare.

1.4 Mental Model

Every element points to a "parent," ultimately tracing up to a **root** representing its set. `find(x)` walks up the parent chain to the root; `union(x, y)` finds both roots and links one under the other. The two optimizations — **path compression** (flatten the chain during `find` so future lookups are faster) and **union by rank/size** (always attach the smaller tree under the bigger one's root, keeping trees shallow) — are what make this fast.

1.5 Visual Explanation

```
Initial: each element is its own set
1  2  3  4  5   (parent[i] = i for all)

union(1,2): parent[find(2)] = find(1) → 1 is now parent of 2
   1
   |
   2     3  4  5

union(3,4): 3 becomes parent of 4
   1        3
   |        |
   2        4     5

union(2,3): find(2)=1, find(3)=3(assuming union by size links 3's root under 1, or vice versa)
   1
  / \
 2   3
     |
     4        5
```

```
find(4): 4 → 3 → 1 (root=1), with path compression, 4's parent becomes 1 directly for future O(1) lookups
```

1.6 Simple Analogy

Union-Find is like **tracing "who's your ultimate boss" in a corporate hierarchy that flattens itself every time you ask** — the first time you trace up a long chain, you "remember" the shortcut directly to the top (path compression), so every future question about anyone in that chain is instant.

1.7 When Should I Immediately Think About Using This Pattern?

- "Number of connected components" with **edges added incrementally** (not a static graph given all at once — though it works there too).
- "Are these two elements connected/in the same group?" repeated many times.
- **Cycle detection in an undirected graph** as edges are processed one by one.
- "Minimum Spanning Tree" (Kruskal's algorithm uses Union-Find directly).
- "Accounts merge," "friend circles," "redundant connection" style grouping problems.

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
"connected components," "number of provinces"	Union-Find (or BFS/DFS — both valid)
"redundant connection"	Cycle detection via Union-Find
"accounts merge"	Grouping via Union-Find
"minimum spanning tree"	Kruskal's algorithm (Union-Find core component)
"are X and Y connected" (repeated queries, dynamic edges)	Union-Find's specific strength
"smallest string with swaps"	Union-Find for grouping swappable positions

2.2 Hidden Hints

Edges/relationships given as a **list of pairs to process incrementally** (rather than a full adjacency list upfront) is a strong signal — Union-Find is specifically designed for this incremental-edge-processing scenario.

2.3 Interview Clues

Interviewer emphasizes "as edges are added one at a time" or "many queries of the form 'are these connected' " — both point toward Union-Find's specific efficiency advantage over repeated BFS/DFS.

2.4 Common Trick Words

"Merge," "group," "province," "circle," "redundant" — these all imply a grouping/connectivity structure that Union-Find directly models.

2.5 What Interviewers Expect

Correct implementation of **both** optimizations (path compression AND union by rank/size) — using only one gives O(log n) instead of the near-O(1) amortized bound, a distinction interviewers may probe. Also expected: recognizing when Union-Find is preferable to BFS/DFS (incremental/dynamic edge addition) versus when a single static BFS/DFS pass is equally valid and simpler.

2.6 When NOT To Use This Pattern

- The graph is **static** (given all at once) and you only need **one-time** connectivity analysis — a single BFS/DFS pass is equally correct and arguably simpler to reason about.
- You need to **remove** edges/elements from a set (Union-Find's classic form doesn't support efficient "split" operations — only merging).
- You need the **actual path** between two connected nodes, not just "are they connected" — Union-Find doesn't retain path information, only set membership.
- The graph is **directed** — Union-Find is fundamentally for undirected connectivity/grouping.

SECTION 3 — Decision Framework

```
Do you need to process EDGES INCREMENTALLY and answer "are these connected" repeatedly?

       Yes → USE UNION-FIND (near-O(1) amortized per operation)

        No

Is the graph STATIC (given all at once) with only ONE connectivity pass needed?

       Yes → PLAIN BFS/DFS is equally correct and often simpler (Pattern #18)

        No

Do you need to detect a CYCLE as UNDIRECTED edges are added one at a time?

       Yes → USE UNION-FIND (union returns "already connected" = cycle detected)

        No

Do you need a MINIMUM SPANNING TREE?

       Yes → USE UNION-FIND as the core of KRUSKAL'S ALGORITHM

        No

Do you need to REMOVE elements from a set, or need DIRECTED connectivity?

       Yes → Union-Find does NOT support this efficiently - reconsider using BFS/DFS or a different struct
```

Why: Union-Find's specific advantage is amortized near-O(1) **incremental** connectivity — for one-shot static connectivity questions, a single BFS/DFS pass achieves the same O(V+E) result with less implementation complexity; Union-Find pulls ahead specifically when connectivity queries and edge additions are interleaved many times.

SECTION 4 — Why This Pattern Works

Mathematical/Logical (Union by Rank/Size): Always attaching the smaller (by rank/height or by size) tree under the larger one's root guarantees that a tree's height can only increase when two trees of **equal** rank merge — meaning height grows at most logarithmically with the number of union operations, giving O(log n) per `find` even without path compression.

Mathematical/Logical (Path Compression): During `find(x)`, redirecting every node along the path directly to the root (instead of just returning the root) means future `find` calls on those nodes are O(1) — this "flattening" effect compounds across many operations.

Combined effect (Amortized Analysis): When both optimizations are used together, a rigorous amortized analysis (using the inverse Ackermann function $\alpha(n)$) shows that a sequence of `m` union/find operations on `n` elements takes $O(m \cdot \alpha(n))$ total time — and since $\alpha(n) \leq 4$ for any `n` up to numbers vastly larger than the number of atoms in the observable universe, this is **effectively O(1) per operation** in any practical scenario.

Correctness Proof (connectivity): *Invariant:* `find(x) == find(y)` if and only if `x` and `y` are in the same connected component, considering only the union operations performed so far. *Base case:* initially, every element is its own singleton set, and `find(x) == find(y)` only when `x == y` — trivially correct. *Inductive step:* `union(x, y)` merges the two sets containing `x` and `y` by linking one root under the other; this correctly makes `find` return the same root for every element in either original set, while leaving all other sets' roots unaffected. *Termination:* after all unions are processed, `find` correctly reflects the transitive closure of all union operations performed. **QED.**

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework

1. Initialize `parent[i] = i` for all elements (each is its own set root); initialize `rank[i] = 0` (or `size[i] = 1`).
2. **find(x):** if `parent[x] != x`, recursively find the root of `parent[x]` and set `parent[x]` directly to that root (path compression); return the root.
3. **union(x, y):** find the roots of `x` and `y`; if different, attach the smaller-rank (or smaller-size) root under the larger one's root (union by rank/size), updating rank/size accordingly.
4. To check connectivity: `find(x) == find(y)`.

5.2 General Template

```
class UnionFind:
    parent = array where parent[i] = i for all i
    rank = array of zeros

    function find(x):
        if parent[x] != x:
            parent[x] = find(parent[x])          # PATH COMPRESSION
        return parent[x]

    function union(x, y):
        rootX = find(x)
        rootY = find(y)
        if rootX == rootY:
            return false                          # already connected (cycle detected, if this was a new e

        if rank[rootX] < rank[rootY]:
            parent[rootX] = rootY
```

```
        else if rank[rootX] > rank[rootY]:
            parent[rootY] = rootX
        else:
            parent[rootY] = rootX
            rank[rootX] = rank[rootX] + 1
        return true                                  # successfully merged (no cycle)
```

5.3 Cycle Detection in Undirected Graph Template

```
function hasCycle(edges, n):
    uf = new UnionFind(n)
    for (u, v) in edges:
        if not uf.union(u, v):                     # union returns false if already connected
            return true                             # cycle detected
    return false
```

5.4 Interview Thinking Process

1. "Since edges are processed incrementally and I need repeated connectivity queries, I'll use Union-Find for near-O(1) amortized operations, rather than repeated BFS/DFS."
2. "I'll implement both path compression (in `find`) and union by rank/size (in `union`) — using only one gives O(log n), not the near-O(1) amortized bound the combination achieves."
3. "For cycle detection, I'll recognize that `union` returning 'already connected' means the new edge would create a cycle."
4. "I'll state the amortized complexity as O(α(n)) per operation, explaining that α(n) is effectively constant for all practical input sizes."

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case (both optimizations)	O(α(n)) amortized per operation	O(n) for parent/rank arrays	Path compression + union by rank bound the amortized cost via inverse Ackermann analysis
Worst Case (path compression only)	O(log n) amortized per operation	O(n)	Trees can still grow to O(log n) height without union-by-rank bounding merges
Worst Case (union by rank only, no compression)	O(log n) per operation	O(n)	Height bounded to O(log n), but no further flattening without compression
Worst Case (neither optimization)	O(n) per operation (degenerate chain)	O(n)	Without either optimization, trees can degrade to linked-list-like chains
Amortized (both optimizations, m operations)	O(m · α(n)) total, α(n) ≤ 4 practically	O(n)	Classic result from amortized analysis literature (Tarjan)

SECTION 7 — Edge Cases

Edge Case	Example	Handling
Single element	`n=1`	Trivially its own set, `find(0) == 0`
No union operations performed	All elements isolated	Every element remains its own singleton set
Union of already-connected elements	`union(x,y)` where `find(x)==find(y)`	Correctly returns false/no-op — this IS the cycle-detection signal in undirected graphs
Self-union	`union(x, x)`	`find(x) == find(x)` trivially — correctly a no-op, not an error
Very large n with many union operations	n=10^6, m=10^6 unions	With both optimizations, remains fast (near-linear total time); without them, risks severe degradation
Forgetting to initialize rank/size arrays	Rank array not properly zeroed	Leads to incorrect union-by-rank decisions — always explicitly initialize
0-indexed vs 1-indexed elements	Problem gives 1-indexed nodes	Ensure parent/rank array sizing and indexing consistently matches the problem's indexing convention

Common mistakes: implementing `find` without path compression (still correct, but loses the near-O(1) amortized guarantee); implementing `union` without checking/using rank or size (always attaching arbitrarily, risking O(n) degenerate chains); off-by-one errors when the problem uses 1-indexed nodes but the array is 0-indexed.

SECTION 8 — Pros & Cons

Advantages: Near-O(1) amortized time for both `find` and `union` with both optimizations; extremely simple, compact implementation (typically under 20 lines); ideal for incremental/dynamic connectivity scenarios. **Disadvantages:** Doesn't support efficient "split"/removal operations; doesn't retain actual path information (only set membership); not applicable to directed graphs. **Trade-offs:** Union-Find (near-O(1) amortized, incremental) vs. BFS/DFS (O(V+E) per full traversal, better for one-shot static analysis or when path information is needed) — choose based on whether connectivity queries are interleaved with edge additions. **Limitations:** Fundamentally an undirected-connectivity tool; augmenting it (e.g., for weighted union or additional metadata per set) is possible but adds implementation complexity. **Inefficient when:** used for a single, one-time static connectivity check — the setup overhead isn't justified versus a single BFS/DFS pass.

SECTION 9 — Real World Applications

Domain	Application
Networking	Dynamic network connectivity monitoring (detecting when previously-connected network segments become partitioned/reconnected)
Databases	Merging duplicate records/entities incrementally (e.g., "Accounts Merge" — combining accounts sharing common emails)
Image Processing	Connected-component labeling in image segmentation (grouping adjacent similar pixels efficiently)
Kruskal's MST Algorithm	Used in network design (minimum-cost cabling/wiring layouts), circuit design
Social Networks	Incremental friend-group/community detection as connections are added over time
Percolation Theory (Physics/Simulation)	Determining if a system "percolates" (top-to-bottom connectivity) as sites are incrementally activated
Version Control / Merge Systems	Detecting and merging equivalent branches/objects incrementally
Compiler Register Allocation	Union-Find used in certain register-coalescing algorithms during compilation
Distributed Systems	Cluster membership and partition detection in gossip-based systems
Game Development	Grouping connected game-world regions/tiles dynamically (e.g., "is this area still reachable")

SECTION 10 — Interview Strategy

How seniors answer: They immediately recognize the incremental-edge-processing signal, implement both optimizations (path compression AND union by rank/size) without prompting, and state the O(α(n)) amortized complexity with the inverse-Ackermann justification, contrasting it explicitly with a one-shot BFS/DFS alternative.

How juniors answer: They often implement Union-Find with only one optimization (or neither), unknowingly settling for O(log n) or O(n) instead of the intended near-O(1) amortized bound, or they default to BFS/DFS even when the incremental nature of the problem makes Union-Find the more natural and efficient fit.

Typical follow-ups: "What's the complexity without path compression?" "What's the complexity without union by rank?" "How would you use this for Kruskal's minimum spanning tree algorithm?" "Can Union-Find handle directed graphs?" (No — explain why, and what would be needed instead).

Optimization questions: "Can you augment Union-Find to also track the size of each connected component efficiently?" (Yes — maintain a `size` array alongside `parent`, updating it during `union`).

SECTION 11 — Pattern Variations

Variation	Description	Example
Basic Union-Find	Core find/union with both optimizations	Number of Connected Components in an Undirected Graph
Cycle Detection (Undirected)	Union returning false signals a cycle	Redundant Connection
Weighted Union-Find	Tracks additional relationship metadata (e.g., ratios) between elements	Evaluate Division (alternative to graph DFS)
Union-Find with Size Tracking	Augmented to answer "size of this element's component"	Number of Islands II (dynamic connectivity)
Kruskal's MST	Union-Find as the cycle-avoidance mechanism while adding edges in weight order	Min Cost to Connect All Points
Grouping/Equivalence Problems	Union-Find models "these items are interchangeable/equivalent"	Accounts Merge, Smallest String With Swaps
Percolation / Grid Connectivity	Union-Find over grid cells with dynamic activation	Number of Islands II, Bricks Falling When Hit

---

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Graph BFS/DFS	O(V+E) per full traversal; simpler for one-shot static analysis	Static graph, single connectivity pass, or need actual path information
Topological Sort	Handles directed dependency ordering, not undirected grouping	Directed acyclic graphs (DAGs) with ordering constraints
Kruskal's vs Prim's (MST)	Kruskal's uses Union-Find; Prim's uses a heap-based greedy expansion instead	Kruskal's is often simpler when edges are naturally processed in sorted order

Comparison Table

Aspect	Union-Find	BFS/DFS
Best for	Incremental/dynamic connectivity queries	One-shot static connectivity
Time per query	O(α(n)) amortized	O(V+E) per full traversal
Supports edge removal	No	N/A (traversal doesn't inherently track edges added incrementally)
Retains path information	No (only set membership)	Yes (can reconstruct paths)

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	Direct connectivity counting	Number of Provinces, Find if Path Exists in Graph
Medium	Cycle detection, grouping/merging	Redundant Connection, Accounts Merge, Number of Connected Components in an Undirected Graph
Hard	Dynamic connectivity, MST construction	Number of Islands II, Min Cost to Connect All Points, Swim in Rising Water
Very Hard	Weighted Union-Find, advanced augmented structures	Evaluate Division, Bricks Falling When Hit, Regions Cut By Slashes

---

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Number of Provinces	Medium	Amazon, Meta, Google	Direct connectivity counting via Union-Find	Foundational mechanics
2	Find if Path Exists in Graph	Easy	Amazon	Basic connectivity check	Basic find/union usage
3	Redundant Connection	Medium	Amazon, Google, Meta	Cycle detection via union returning false	Cycle detection mastery
4	Redundant Connection II	Hard	Google, Amazon	Directed graph variant requiring modified logic	Advanced directed-edge handling
5	Number of Connected Components in an Undirected Graph	Medium	Amazon, Meta, Google	Direct component counting	Component counting reinforcement
6	Accounts Merge	Medium	Amazon, Meta, Google	Grouping via Union-Find over shared attributes	Grouping/equivalence application
7	Smallest String With Swaps	Medium	Amazon, Google	Grouping swappable indices via Union-Find	Index-grouping application

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
8	Number of Islands II	Hard	Amazon, Google	Dynamic connectivity as land is added incrementally	Advanced dynamic connectivity
9	Min Cost to Connect All Points	Medium	Amazon, Google	Kruskal's MST using Union-Find	MST construction mastery
10	Swim in Rising Water	Hard	Google, Amazon	Union-Find combined with sorted edge processing	Advanced sorted-edge Union-Find
11	Evaluate Division	Medium	Amazon, Google, Meta	Weighted Union-Find (or graph DFS alternative)	Weighted Union-Find application
12	Satisfiability of Equality Equations	Medium	Amazon, Google	Union-Find for equivalence class checking	Equivalence-class application
13	Regions Cut By Slashes	Medium	Google, Amazon	Union-Find over subdivided grid cells	Advanced grid-based Union-Find
14	Bricks Falling When Hit	Hard	Google, Amazon	Reverse-time Union-Find simulation	Advanced reverse-processing technique
15	Most Stones Removed with Same Row or Column	Medium	Amazon, Google	Union-Find for grouping shared-row/column stones	Grouping via shared attribute
16	Optimize Water Distribution in a Village	Hard	Google, Amazon	Union-Find combined with virtual source node for MST	Advanced MST with virtual nodes
17	Making A Large Island	Hard	Amazon, Google	Union-Find (or DFS) for island size computation with one flip allowed	Advanced grid Union-Find
18	Graph Valid Tree	Medium	Google, Amazon	Union-Find for cycle detection + connectivity combined check	Cross-pattern (cycle + connectivity)

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
19	Longest Consecutive Sequence (Union-Find alternative)	Medium	Amazon, Meta, Google	Union-Find alternative to the standard hash-set approach	Alternative technique comparison
20	Couples Holding Hands	Hard	Google, Amazon	Union-Find for minimum-swap cycle counting	Advanced cycle-counting application
21	Similar String Groups	Hard	Google, Amazon	Union-Find for grouping similar strings	Advanced grouping application
22	Sentence Similarity II	Medium	Google, Amazon	Union-Find for word-equivalence grouping	Equivalence-class application
23	Minimize Malware Spread	Hard	Google, Amazon	Union-Find for component-based impact analysis	Advanced component-impact analysis
24	Minimize Malware Spread II	Hard	Google	Advanced Union-Find with removal-impact analysis	Advanced impact analysis
25	Checking Existence of Edge Length Limited Paths	Hard	Google, Amazon	Offline Union-Find with sorted queries	Advanced offline query processing
26	Remove Max Number of Edges to Keep Graph Fully Traversable	Hard	Google	Dual Union-Find (Alice/Bob) with shared-edge prioritization	Advanced dual-structure Union-Find
27	Path With Minimum Effort (contrast, heap-based)	Medium	Google, Amazon	Contrast: Union-Find alternative to Dijkstra’s-style approach	Alternative technique comparison
28	The Earliest Moment When Everyone Become Friends	Medium	Google, Amazon	Union-Find with sorted-time edge processing	Sorted-time processing application

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
29	Number of Operations to Make Network Connected	Medium	Amazon, Google	Union-Find for counting redundant edges and components	Component + redundant-edge counting
30	Last Day Where You Can Still Cross	Hard	Google	Reverse-time Union-Find (grid connectivity)	Advanced reverse-processing grid application

---

SECTION 15 — Common Mistakes

1. Implementing `find` without path compression, losing the near-O(1) amortized guarantee (settling for O(log n) instead). *Fix:* always redirect nodes along the find path directly to the root.
2. Implementing `union` without rank/size comparison, risking degenerate O(n) chains. *Fix:* always attach the smaller tree under the larger one's root.
3. Forgetting that `union` returning "already connected" (same root) is exactly the cycle-detection signal for undirected graphs — missing this shortcut and writing separate, redundant cycle-detection logic. *Fix:* directly use `union`'s return value for cycle detection.
4. Off-by-one errors when the problem's node indexing doesn't match the array's indexing (1-indexed vs 0-indexed). *Fix:* explicitly verify and adjust indexing consistently.
5. Attempting to use Union-Find for directed graphs or for operations requiring element removal/splitting from a set, which it doesn't natively support. *Fix:* recognize these as signals to use a different technique (DFS-based directed cycle detection, or a different data structure entirely).

Why people fail: implementing "a" Union-Find (with some optimization) is common knowledge, but implementing the **fully optimized** version (both path compression AND union by rank/size) correctly, and being able to articulate *why* the combination gives the striking O(α(n)) bound rather than just O(log n), is what separates surface-level familiarity from deep understanding — many candidates can write working code but can't defend the precise complexity claim.

SECTION 16 — Optimization Techniques

- **Time:** Always implement both path compression and union by rank/size together — using only one is a common, easily-avoidable regression from near-O(1) to O(log n).
- **Space:** O(n) for parent and rank/size arrays is already minimal and rarely worth optimizing further.
- **Readability:** Encapsulate Union-Find as a small, reusable class/struct with clearly named `find` and `union` methods — this is one of the most reusable, "memorize once, apply everywhere" interview data structures.
- **Interview performance:** Explicitly state both optimizations and their combined amortized complexity (O(α(n)) via inverse Ackermann function) — this precise articulation is a strong differentiator.

SECTION 17 — Multiple Language Templates

Java

```java
class UnionFind {
    int[] parent, rank;
    UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    boolean union(int x, int y) {
        int rootX = find(x), rootY = find(y);
        if (rootX == rootY) return false;
        if (rank[rootX] < rank[rootY]) { int t = rootX; rootX = rootY; rootY = t; }
        parent[rootY] = rootX;
        if (rank[rootX] == rank[rootY]) rank[rootX]++;
        return true;
    }
}
```

JavaScript

```javascript
class UnionFind {
    constructor(n) {
        this.parent = Array.from({length: n}, (_, i) => i);
        this.rank = new Array(n).fill(0);
    }
    find(x) {
        if (this.parent[x] !== x) this.parent[x] = this.find(this.parent[x]);
        return this.parent[x];
    }
    union(x, y) {
        let rootX = this.find(x), rootY = this.find(y);
        if (rootX === rootY) return false;
        if (this.rank[rootX] < this.rank[rootY]) [rootX, rootY] = [rootY, rootX];
        this.parent[rootY] = rootX;
        if (this.rank[rootX] === this.rank[rootY]) this.rank[rootX]++;
        return true;
    }
}
```

PHP

```php
class UnionFind {
    private array $parent, $rank;
    public function __construct(int $n) {
        $this->parent = range(0, $n - 1);
        $this->rank = array_fill(0, $n, 0);
    }
    public function find(int $x): int {
```

```
        if ($this->parent[$x] !== $x) $this->parent[$x] = $this->find($this->parent[$x]);
        return $this->parent[$x];
    }
    public function union(int $x, int $y): bool {
        $rootX = $this->find($x); $rootY = $this->find($y);
        if ($rootX === $rootY) return false;
        if ($this->rank[$rootX] < $this->rank[$rootY]) [$rootX, $rootY] = [$rootY, $rootX];
        $this->parent[$rootY] = $rootX;
        if ($this->rank[$rootX] === $this->rank[$rootY]) $this->rank[$rootX]++;
        return true;
    }
}
```

Python

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        root_x, root_y = self.find(x), self.find(y)
        if root_x == root_y:
            return False
        if self.rank[root_x] < self.rank[root_y]:
            root_x, root_y = root_y, root_x
        self.parent[root_y] = root_x
        if self.rank[root_x] == self.rank[root_y]:
            self.rank[root_x] += 1
        return True
```

Go

```
type UnionFind struct {
    parent, rank []int
}
func NewUnionFind(n int) *UnionFind {
    parent := make([]int, n)
    for i := range parent {
        parent[i] = i
    }
    return &UnionFind{parent: parent, rank: make([]int, n)}
}
func (uf *UnionFind) Find(x int) int {
    if uf.parent[x] != x {
        uf.parent[x] = uf.Find(uf.parent[x])
    }
    return uf.parent[x]
}
```

```go
func (uf *UnionFind) Union(x, y int) bool {
    rootX, rootY := uf.Find(x), uf.Find(y)
    if rootX == rootY {
        return false
    }
    if uf.rank[rootX] < uf.rank[rootY] {
        rootX, rootY = rootY, rootX
    }
    uf.parent[rootY] = rootX
    if uf.rank[rootX] == uf.rank[rootY] {
        uf.rank[rootX]++
    }
    return true
}
```

C++

```cpp
class UnionFind {
    vector<int> parent, rank_;
public:
    UnionFind(int n) : parent(n), rank_(n, 0) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    bool unite(int x, int y) {
        int rootX = find(x), rootY = find(y);
        if (rootX == rootY) return false;
        if (rank_[rootX] < rank_[rootY]) swap(rootX, rootY);
        parent[rootY] = rootX;
        if (rank_[rootX] == rank_[rootY]) rank_[rootX]++;
        return true;
    }
};
```

SECTION 18 — Dry Runs

Small Input

`n=5`, edges: `(0,1)`, `(1,2)`, `(3,4)`, `(2,3)` — check for cycle and final component count

`Initial: parent=[0,1,2,3,4], rank=[0,0,0,0,0]`

`union(0,1): find(0)=0, find(1)=1, different → rank equal → parent[1]=0, rank[0]=1`
`parent=[0,0,2,3,4]`

`union(1,2): find(1)=0 (via parent), find(2)=2, different → rank[0]=1 > rank[2]=0 → parent[2]=0`
`parent=[0,0,0,3,4]`

`union(3,4): find(3)=3, find(4)=4, different → rank equal → parent[4]=3, rank[3]=1`
`parent=[0,0,0,3,3]`

```
union(2,3): find(2)=0, find(3)=3, different → rank[0]=1 == rank[3]=1 → parent[3]=0, rank[0]=2
   parent=[0,0,0,0,3] → find(4) would now also resolve to 0 (via 3)

No cycle detected (all unions succeeded); final component count = 1 (all connected)
```

Large Input (Conceptual)

For n=10^6 elements and m=10^6 union operations, with both optimizations the total time is $O(m \cdot \alpha(n)) \approx O(4 \times 10^6)$ — effectively linear, versus $O(m \times n) = O(10^{12})$ for a naive approach without any optimization (repeatedly re-scanning to find set membership).

Corner Case

`union(x, x)` (self-union): `find(x) == find(x)` trivially → returns false immediately, correctly a no-op without any incorrect state change.

SECTION 19 — Advanced Concepts

- **Weighted Union-Find (Evaluate Division):** augmenting each element with a "weight" relative to its parent (e.g., a ratio) allows answering quantitative relationship queries (e.g., "what is a/b given a/c and c/b are known") using the same union/find structure, updating weights appropriately during path compression.
- **Reverse-time Union-Find (Bricks Falling When Hit, Last Day Where You Can Still Cross):** for problems framed as "removals" (which Union-Find can't handle directly), process the removal events in **reverse order**, turning each "removal" into an "addition" (union), and reverse the final answer sequence — a clever, broadly applicable trick for adapting Union-Find to seemingly incompatible removal-based problems.
- **Union-Find with virtual nodes (Optimize Water Distribution):** introducing an artificial "virtual source" node connected to every real node with a cost equal to that node's individual well-cost transforms certain optimization problems into a standard Kruskal's MST problem solvable with Union-Find.
- **Kruskal's Algorithm:** sort all edges by weight, then process them in increasing order, using Union-Find's `union` to add an edge to the MST only if it doesn't create a cycle (i.e., `union` returns true) — directly demonstrating Union-Find as a core building block of a classic greedy MST algorithm.

SECTION 20 — Senior Engineer Insights

Staff engineers recognize Union-Find as the go-to structure whenever a problem involves **incremental grouping/merging with repeated "same group?" queries** — a pattern that recurs far beyond graph problems: deduplicating records with transitive equivalence relationships (if A matches B and B matches C, then A matches C), tracking evolving cluster membership in distributed systems, and detecting emergent cycles as configuration dependencies are added dynamically. They also know the "reverse-time" trick for adapting Union-Find to removal-based problems it doesn't natively support, and can quickly derive the weighted-union variant for quantitative relationship problems. Interviewers evaluate whether a candidate implements *both* optimizations correctly and can precisely articulate the resulting near-constant amortized complexity, rather than treating Union-Find as a black-box template.

SECTION 21 — Cheat Sheet

```
PATTERN: Union-Find (Disjoint Set Union)
RECOGNIZE: incremental edge processing, "connected components," "redundant connection," "merge accounts,"
TEMPLATE:
    parent[i] = i for all i; rank[i] = 0
    find(x): if parent[x] != x: parent[x] = find(parent[x])  # path compression
             return parent[x]
    union(x, y): rootX, rootY = find(x), find(y)
                 if rootX == rootY: return false              # cycle / already connected
                 attach smaller-rank root under larger-rank root; update rank if equal
                 return true
COMPLEXITY: O( (n)) amortized per operation (both optimizations combined) - effectively O(1)
KEY PROOF: path compression flattens trees; union by rank bounds height growth - combined, amortized analy
WATCH FOR: implementing BOTH optimizations (not just one), union's false-return as the cycle-detection sig
DOESN'T APPLY WHEN: directed graphs, need element removal/split, one-shot static connectivity (plain BFS/D
```

SECTION 22 — Revision Notes (5-Minute Review)

- Union-Find: `find` (path compression) + `union` (by rank/size) = O(α(n)) amortized, effectively O(1).
- `union` returning false (same root already) = cycle detected in undirected graph — no separate cycle check needed.
- Best for incremental/dynamic edge processing with repeated connectivity queries; plain BFS/DFS suffices for one-shot static analysis.
- Core of Kruskal's MST algorithm (sort edges, union if no cycle).
- Doesn't support directed graphs or element removal — use the reverse-time trick to adapt removal-based problems.
- Weighted Union-Find extends this to quantitative relationship queries (Evaluate Division).

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Basic find/union mechanics	Number of Provinces (547), Find if Path Exists in Graph (1971)
Intermediate	Cycle detection, grouping	Redundant Connection (684), Accounts Merge (721), Smallest String With Swaps (1101)
Advanced	MST construction, dynamic connectivity	Min Cost to Connect All Points (1584), Number of Islands II (305)
Expert	Weighted Union-Find, reverse-time tricks	Evaluate Division (399), Bricks Falling When Hit (803), Similar String Groups (839)

SECTION 24 — Memory Tricks

- **Mnemonic:** "**F**latten and **A**ttach Small-to-Big" (FASB) — Flatten via path compression, Attach smaller tree under bigger.
- **Visualization: Company acquisitions** — everyone eventually reports to the ultimate parent company, and the reporting chain shortens itself every time someone asks "who's my ultimate boss?"
- **Recognition shortcut:** "Incremental edges" + "are these connected" (repeated) → Union-Find, implement both optimizations.

SECTION 25 — Final Summary

Union-Find maintains disjoint sets with near-O(1) amortized `find` and `union` operations by combining path compression (flattening lookup chains) with union by rank/size (bounding tree height growth) — a combination whose amortized complexity is governed by the inverse Ackermann function, effectively constant for any practical input. The single most important thing to remember forever: **always implement both optimizations together, and recognize that `union` returning "already connected" is exactly the cycle-detection signal for undirected graphs — no separate cycle-checking logic is needed.** Union-Find shines specifically for incremental, dynamic connectivity scenarios; for one-shot static connectivity questions, a single BFS/DFS pass is equally correct and often simpler.